CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

PROGRAMMING WITH C++

UNIT 4

INTRODUCTION TO CLASSES AND INHERITANCE

DEPSTAR

Devang Patel Institute of Advance Technology and Research

Outlines

- Class and object
- Set and Get Member Functions
- Custom Constructors
- Class Scope and Accessing Class Members
- Access Functions and Utility Functions
- Destructors
- Default Assignment Operator
- Const Objects and Const Member Functions
- Composition: Objects as Members of Classes
- Friend Classes
- Aggregation
- Base Classes and Derived Classes
- Types of inheritance
- Public, private and protected in Inheritance
- Constructors and Destructors in Derived Classes

Class and object

- C++ is an object-oriented programming (OOP) language, which means that it is built around the concept of objects and classes.
- These two are key concepts that help you organize and manage data and behavior in a more structured way.

Class

- A class in C++ is a user-defined blueprint or template for creating objects. It defines the properties (data members) and actions (member functions) that its objects will have.
    - A class allows you to bundle data and functions that operate on that data together.
    - The class itself is not a real entity; it’s just a definition. The object is what holds actual values and resides in memory.

Class Syntax

Syntax:

```
class ClassName {
public:
   // Data members (attributes or properties)
   int attribute1;
   double attribute2;

   // Member functions (methods or behaviors)
   void method1() {
      // Function code
   }

   void method2() {
      // Function code
   }
};
```

Components of a Class

- **Data Members**: These are the variables declared inside a class. They represent the attributes or state of the object.

- **Member Functions**: Functions inside the class that define the actions an object can perform. These can access and manipulate the data members of the class.

- **Access Specifiers**: Determine the visibility of members of the class. The most common ones are
  - **public**: Members are accessible from outside the class.
  - **private**: Members are only accessible from within the class.
  - **protected**: Members are accessible within the class and derived classes.

- **Constructor**: A special member function that is called when an object is created. It is used to initialize the object's data members.

- **Destructor**: A special member function that is called when an object is destroyed (i.e., when it goes out of scope or is explicitly deleted)

Object

- An **object** is an instance of a class. While a class is just a blueprint, an object is a specific, created entity that occupies memory.
- Creating an object from a class essentially means allocating memory for it and initializing it with default or user-specified values.

Syntax:

```
ClassName objectName;
```

Example of Class and Object

```
#include <iostream>
using namespace std;

// Class definition
class Car {
public:
    // Data members
    string make;
    string model;
    int year;

    // Constructor to initialize the object
    Car(string m, string mo, int y) {
        make = m;
        model = mo;
        year = y;
    }
```

```
 // Member function to display car details
    void displayDetails() {
        cout << "Car Make: " << make << endl;
        cout << "Car Model: " << model << endl;
        cout << "Car Year: " << year << endl;
    }
};

int main() {
    // Create an object of class 'Car'
    Car car1("Toyota", "Corolla", 2020);  // Constructor is called here

    // Access and use the member function of the object
    car1.displayDetails();

    return 0;
}
```

Output:

```
Car Make: Toyota
Car Model: Corolla
Car Year: 2020
```

Core Concepts of Classes and Objects

Data Members (Attributes)

- Data members are the variables inside a class that hold information about the object.
- They represent the state of an object, and their values are specific to each object.

Example:

```
class Car {
public:
        string make;
        string model;
        int year;
};
```

Core Concepts of Classes and Objects

Member Functions (Methods)

- Member functions are the operations or behaviors that objects of the class can perform.
- These functions can modify or access the data members.

Example:

```
// Member function to display car details
void displayDetails() {
        cout << "Car Make: " << make << endl;
        cout << "Car Model: " << model << endl;
        cout << "Car Year: " << year << endl;
}
```

Core Concepts of Classes and Objects

Constructors and Destructors

- **Constructor**: A **constructor** is a special member function that is automatically called when an object is created**.** Its main role is to initialize the object's data members**.**
  - **Default Constructor**: A constructor with no parameters.
  - **Parameterized Constructor**: A constructor that accepts parameters to initialize the object with specific values.

Example:

```
class Car
{
public:
   string make;
   string model;
   int year;

   // Default constructor
   Car() {
      make = "Unknown";
      model = "Unknown";
      year = 0;
   }
   // Parameterized constructor
      Car(string m, string mo, int y) {
         make = m;
         model = mo;
         year = y;
      }
};
```

Core Concepts of Classes and Objects

Destructor

- A destructor is called when an object is destroyed (goes out of scope). It is used for cleaning up resources (like closing files, freeing memory) that the object may have acquired during its lifetime.

Example:

```
class Car {
public:
   // Constructor
   Car() {
      cout << "Car created!" << endl;
   }

   // Destructor
   ~Car() {
      cout << "Car destroyed!" << endl;
   }
};
```

Set and Get Member Functions

- In C++, set and get functions (also known as setter and getter functions) are commonly used to access and modify the private data members of a class.

- These functions provide controlled access to the attributes of a class, following the principle of encapsulation in object-oriented programming.

- By using set and get functions, we can ensure that the data is accessed in a safe and controlled manner, and we can enforce rules or validations when setting values.

advantages of using setter and getter functions in C++

Encapsulation

- **Data Hiding**: Setters and getters help hide the internal implementation of a class. By making the data members private and providing public setter and getter functions, you can control how data is accessed and modified. This ensures that internal details are hidden from outside code, which is a core principle of object-oriented programming.

- **Control Over Data**: Encapsulation provides the ability to modify or validate the data before it's accessed or changed, preventing direct manipulation of sensitive class attributes.

Getter Function (Get Function)

A getter function is used to retrieve the value of a private data member. It allows read-only access to the class attributes, meaning the external code can access the value but cannot modify it directly.

Syntax:

```
data_type getMemberName()
{
    //Body
};
```

- data_type: The type of the member variable.
- getMemberName: The function name, typically following the pattern get followed by the member variable's name.

Setter Function (Set Function)

A setter function is used to modify the value of a private data member. It allows write access to the class attributes, and it can include validation or checks before setting the value.

Syntax:

```
void setMemberName(data_type value)
{
    //Body
};
```

- data_type: The type of the member variable.
- setMemberName: The function name, typically following the pattern set followed by the member variable's name.

Example of Get and Set

```
#include <iostream>
#include <string>
using namespace std;

class Person {
private:
    string name;   // Private data member
    int age;       // Private data member

public:
    // Getter function for name
    string getName() const {
        return name;  // Return the name value
    }
    // Setter function for name
    void setName(const string& newName) {
        name = newName;  // Set the name value
    }
    // Getter function for age
    int getAge() const {
        return age;  // Return the age value
    }

    // Setter function for age with validation
    void setAge(int newAge) {
        if (newAge >= 0) {  // Ensure the age is a positive number
            age = newAge;
        } else {
            cout << "Age must be a positive number!" << endl;
        }
    }
};

int main() {
    // Create a Person object
    Person person;

    // Set values using setter functions
    person.setName("John Doe");
    person.setAge(30);

    // Get values using getter functions
    cout << "Name: " << person.getName() << endl;
    cout << "Age: " << person.getAge() << endl;

    // Try to set an invalid age
    person.setAge(-5);  // This will trigger a validation message

    return 0;
}
```

Output:

```
Name: John Doe
Age: 30
Age must be a positive number!
```

Constructor

- A constructor in C++ is a special member function that is automatically called when an object of a class is created.

- Its primary purpose is to initialize the object's data members and allocate any necessary resources.

- Constructors are fundamental to object-oriented programming in C++ because they help ensure that objects are in a valid and well-defined state when they are created.

Types of Constructor:

- Default constructor
- Parameterized constructor
- Copy constructor

Definition of a Constructor

A constructor has the following key properties:

- **Same name as the class**: The constructor's name is always identical to the class name.
- **No return type**: A constructor does not return any value, not even void.
- **Automatically called**: It is invoked automatically when an object is instantiated.
- **Can be overloaded**: You can have multiple constructors in a class with different parameter lists (constructor overloading).

Syntax:

```
class ClassName {
public:
    ClassName() {
        // Constructor code here
    }
};
```

Default Constructor

- A default constructor is one that takes no arguments. If you don't explicitly define any constructor, C++ provides a default constructor automatically.

Example:

```
class MyClass {
public:
    MyClass() {
        // Default constructor code
        std::cout << "Default constructor called!" << std::endl;
    }
};

//When Create an Object
MyClass obj; // Calls the default constructor
```

Parameterized Constructor

- A parameterized constructor is a constructor that takes one or more parameters. It allows you to initialize an object with specific values when it is created.

Example:

```
class MyClass {
private:
   int value;
public:
   MyClass(int v) {  // Constructor with parameter
      value = v;
   }

   void showValue() {
      std::cout << "Value: " << value << std::endl;
   }
};


//When create an object
MyClass obj(10);  // Calls parameterized constructor with '10'
obj.showValue();  // Output: Value: 10
```

Copy Constructor

- A copy constructor is a special constructor used to create a new object as a copy of an existing object.

- C++ provides a default copy constructor if you do not define one, but it performs a shallow copy (just copying the values of data members).

Syntax:

```
ClassName(const ClassName& other);
```

Example of Copy Constructor

Example:

```
class MyClass {
public:
   int x;
   MyClass(int val) : x(val) {}

   // Copy constructor
   MyClass(const MyClass& other) {
      x = other.x;
   }

   void display() {
      std::cout << "x = " << x << std::endl;
   }
};

MyClass obj1(5);
MyClass obj2 = obj1;  // Calls the copy constructor
obj2.display();  // Output: x = 5
```

Constructor Overloading

- In C++, you can overload constructors, meaning you can have multiple constructors with the same name but different parameter lists. The correct constructor is selected based on the arguments passed when the object is created.

Example:

```
class MyClass {
public:
   MyClass() {
      std::cout << "Default constructor" << std::endl;
   }

   MyClass(int x) {
      std::cout << "Parameterized constructor with value " << x << std::endl;
   }

   MyClass(double x) {
      std::cout << "Parameterized constructor with value " << x << std::endl;
   }
};

//When an object create
MyClass obj1;       // Calls default constructor
MyClass obj2(10);    // Calls parameterized constructor (int)
MyClass obj3(10.5);  // Calls parameterized constructor (double)
```

Class Scope and Accessing Class Members

- In C++, class scope refers to the context in which the members (variables and methods) of a class are defined.
- It determines how and where these members can be accessed, modified, or used.
- The accessibility of class members is governed by access specifiers (public, private, and protected).

Access Specifiers:

- **public**: Members declared as public can be accessed from anywhere (outside the class as well as inside).
- **private**: Members declared as private can only be accessed from within the class itself or by its friend functions.
- **protected**: Members declared as protected can be accessed within the class and by classes that inherit from it.

Access Level	Class Member Access	Derived Class Access	Outside Class Access
`public`	Yes	Yes	Yes
`protected`	Yes	Yes	No
`private`	Yes	No	No

Class Definition and Scope

- In C++, a class is defined using the class keyword, and it has a scope that defines its members (variables and functions).
- All members inside a class are defined in that scope, and you can access them using objects of the class, or, in some cases, directly within the class itself.

Example:

```
class MyClass {
public:    // public section
   int publicVar;  // can be accessed outside the class
   void publicMethod() {
      // Can be called from outside the class
      std::cout << "Public Method" << std::endl;
   }

private:   // private section
   int privateVar;  // can only be accessed within the class
   void privateMethod() {
      std::cout << "Private Method" << std::endl;
   }

protected: // protected section
   int protectedVar;  // can be accessed by derived classes
};
```

Example of Accessing Class public Members

Example:

```
class MyClass {
public:
    int publicVar;  // Public member
    void publicMethod() {
        std::cout << "Public Method" << std::endl;
    }
};

int main() {
    MyClass obj;
    obj.publicVar = 10;  // Accessible because it's public
    obj.publicMethod();  // Accessible because it's public
    return 0;
}
```

Example of Accessing Class Private Members

Example:

```
class MyClass {
private:
   int privateVar;  // Private member
   void privateMethod() {
      std::cout << "Private Method" << std::endl;
   }

public:
   void setPrivateVar(int value) { // A public setter
      privateVar = value;
   }
   void getPrivateVar() {
      std::cout << "Private Variable: " << privateVar << std::endl;
   }
};
```

```
int main() {
   MyClass obj;
   // obj.privateVar = 10;  // Error: privateVar is private
   obj.setPrivateVar(10);  // Accessing private variable through public setter
   obj.getPrivateVar();    // Accessing private variable through public getter
   return 0;
}
```

Example of Accessing Class Protected Members

Example:

```
class Base {
protected:
   int protectedVar;
};

class Derived : public Base {
public:
   void accessProtectedVar() {
      protectedVar = 5;  // Can access protected member in derived class
      std::cout << "Protected Variable: " << protectedVar << std::endl;
   }
};

int main() {
   Derived obj;
   obj.accessProtectedVar();  // Access protected member via derived class
   return 0;
}
```

Friend Functions (Accessing Private Members)

- Friend functions allow specific non-member functions to access private and protected members of the class.

- This is useful when you need to access private members but don't want to expose them to the outside world.

Example:

```
class MyClass {
private:
   int privateVar;

public:
   MyClass(int val) : privateVar(val) {}

   friend void friendFunction(MyClass &obj);  // Declare friend function
};

void friendFunction(MyClass &obj) {
   // Access private member directly from friend function
   std::cout << "Private Var: " << obj.privateVar << std::endl;
}

int main() {
   MyClass obj(10);
   friendFunction(obj);  // Friend function can access private member
   return 0;
}
```

Access Functions and Utility Functions

- In C++, **access functions** and **utility functions** are often used to interact with data members of a class or perform common tasks.

  - **Access Functions**
    - Access functions, also known as **getter** and **setter** functions, are used to read (get) or modify (set) the private or protected members of a class. Since C++ supports the concept of **encapsulation**, which keeps data members private to prevent direct access from outside the class, access functions provide a controlled way of accessing and modifying that data.
      - **Types of Access Functions**:
        - **Getter (Accessors):** These functions return the value of private data members.
        - **Setter (Mutators):** These functions set the value of private data members.

  - **Utility Functions**
    - Utility functions are helper functions that provide common tasks or operations that assist with class functionality but are not necessarily related to direct access to data members. These functions are typically used to simplify complex operations, improve readability, or offer convenience.
      - **Utility functions can be**:
        - Functions that perform calculations or transformations.
        - Functions that validate conditions.
        - Functions that manage resources, etc.

Example of Access Function

Example:

```
class Person {
private:
   std::string name;
   int age;

public:
   // Getter (Access function)
   std::string getName() const {
      return name;
   }

   // Setter (Access function)
   void setName(const std::string& newName) {
      name = newName;
   }

   // Getter (Access function)
   int getAge() const {
      return age;
   }

   // Setter (Access function)
   void setAge(int newAge) {
      if (newAge >= 0) { // Simple validation
         age = newAge;
      }
   }
};
```

Example of Utility Function

Example:

```
class Rectangle {
private:
   double width, height;

public:
   Rectangle(double w, double h) : width(w), height(h) {}

   // Utility function: Calculate area
   double calculateArea() const {
      return width * height;
   }

   // Utility function: Calculate perimeter
   double calculatePerimeter() const {
      return 2 * (width + height);
   }

   // Utility function: Check if square
   bool isSquare() const {
      return width == height;
   }
};
```

Destructors

- A **destructor** is a special member function in a class that is called when an object of that class is destroyed or goes out of scope.
- The primary role of a destructor is to release any resources (such as memory or file handles) that were acquired by the object during its lifetime.

Key Characteristics of Destructors:

- **Automatic Call**: A destructor is automatically called when an object goes out of scope or is explicitly deleted (in the case of dynamically allocated memory using new).
- **No Return Type**: Unlike regular functions, destructors do not have a return type, not even void.
- **Same Name as the Class**: A destructor has the same name as the class, preceded by a tilde (~).
- **Cannot Take Arguments**: Destructors do not accept parameters and therefore cannot be overloaded.
- **Called Once**: A destructor is called only once during the lifetime of an object—when the object is destroyed.

Syntax:

```
~ClassName();
```

Example of Destructor

Example:

```
#include <iostream>
class MyClass {
public:
        // Constructor
        MyClass() {
                std::cout << "Constructor called\n";
        }

        // Destructor
        ~MyClass() {
                std::cout << "Destructor called\n";
        }
};

int main() {
        MyClass obj;
        return 0;
}
```

Output:

```
Constructor called
Destructor called
```

Default Assignment Operator

- In C++, the assignment operator (operator=) is used to assign the contents of one object to another object of the same type.

- If you do not explicitly define an assignment operator for a class, the compiler provides a default assignment operator.

- This default operator performs a member-wise assignment, meaning it assigns each member variable of one object to the corresponding member variable of another object.

Example of Default Assignment Operator

Example:

```
#include <iostream>
using namespace std;

class Simple {
public:
   int value;

   // Constructor
   Simple(int v) : value(v) {}

   // Print function
   void print() const {
      cout << "value: " << value << endl;
   }
};

int main() {
   // Create two Simple objects
   Simple obj1(10);
   Simple obj2(20);

   cout << "Before assignment:" << endl;
   cout << "obj1: "; obj1.print();
   cout << "obj2: "; obj2.print();

   // Use default assignment operator
   obj2 = obj1;  // Default assignment operator is called here

   cout << "\nAfter assignment:" << endl;
   cout << "obj1: "; obj1.print();
   cout << "obj2: "; obj2.print();

   return 0;
}
```

Output:

```
Before assignment:
obj1: value: 10
obj2: value: 20

After assignment:
obj1: value: 10
obj2: value: 10
```

Const Objects

- A const object is an object that cannot have its state modified after it is initialized. When you declare an object as const, you are telling the compiler that its value should not be changed.

Syntax: `const MyClass obj;`

- This means that obj is a constant object, and its members cannot be modified through that object.
- If any member of MyClass is non-const, attempting to modify it will result in a compiler error.

Key Points of Const Object

- Const objects can only call const member functions.
- You can initialize a const object through its constructor, but after initialization, its members cannot be changed.
- Const objects are typically used when you want to ensure that the object's data remains read-only after its creation.

Example of Const Objects

Example:

```
class MyClass {
public:
    int value;
    MyClass(int v) : value(v) {}
    void setValue(int v) { value = v; }
    int getValue() const { return value; }
};

int main() {
    const MyClass obj(10);  // const object
    // obj.value = 20; // Error: Cannot modify const object
    std::cout << obj.getValue();  // OK: Can call const member functions
    // obj.setValue(20); // Error: Cannot call non-const function
}
```

Const Member Functions

- A const member function is a function that guarantees it will not modify the state of the object. This is indicated by adding const to the function's declaration after the parameter list.

Syntax:

```
ReturnType functionName() const;
```

- This tells the compiler that this member function does not modify any member variables of the object (except for mutable members), and it can be called on const objects.

Key Points of Const Member Functions

- Const member functions can be called on const objects or const references to objects.
- A const member function can only call other const member functions within it, and cannot modify non-mutable member variables.
- It can be used to guarantee that the method will not change the state of the object, making it safer in multi-threaded or read-only scenarios.

Example of Const Member Function

Example:

```
class MyClass {
public:
   int value;

   MyClass(int v) : value(v) {}

   // Const member function: Guarantees no modification
   int getValue() const {
      return value;
   }

   // Non-const member function: Can modify object state
   void setValue(int v) {
      value = v;
   }
};

int main() {
   const MyClass obj(10);  // const object
   std::cout << obj.getValue();  // OK: Can call const function
   // obj.setValue(20); // Error: Cannot call non-const function
}
```

Composition: Objects as Members of Classes

- In object-oriented programming, composition is a design principle where one class contains objects of other classes as its members.

- This allows you to build complex types by combining simpler ones. Composition is often described as a "has-a" relationship, where one object "has" other objects as part of its structure.

- In C++, composition is implemented by including objects of other classes as data members in a class.

- The lifetime of these objects is controlled by the containing (or "composite") class. Composition is preferred over inheritance when the relationship between objects is better described as "a part of" rather than "is a".

Composition Example

```
#include <iostream>
using namespace std;

class Engine {
public:
   void start() {
      cout << "Engine started!" << endl;
   }

   void stop() {
      cout << "Engine stopped!" << endl;
   }
};

class Car {
private:
   Engine engine;  // Composition: Car "has" an Engine
public:
   void startCar() {
      engine.start();  // Start the engine as part of starting the car
      cout << "Car is now running!" << endl;
   }

   void stopCar() {
      engine.stop();  // Stop the engine as part of stopping the car
      cout << "Car has stopped." << endl;
   }
};

int main() {
   Car myCar;
   myCar.startCar();  // Car starts, and engine starts
   myCar.stopCar();   // Car stops, and engine stops
   return 0;
}
```

Output:

```
Engine started!
Car is now running!
Engine stopped!
Car has stopped.
```

Advantages of Composition

- **Reusability**: Components of the composed class (like Engine in the Car class) can be reused in other contexts (e.g., a Boat might have an Engine).

- **Encapsulation**: It allows for better abstraction, as the internal workings of the composed objects are hidden from the outside world.

- **Flexibility**: Composition provides greater flexibility compared to inheritance because you can easily replace or change the components (objects) of the class.

Friend Classes

- A friend class in C++ is a class that has access to the private and protected members of another class. This allows two classes to work closely together while still maintaining encapsulation.

- **Syntax of Friend Class**

  A class can declare another class as a **friend** using the friend keyword.

Example

```
#include <iostream>
using namespace std;

class B; // Forward declaration

class A {
private:
   int numA;

public:
   A(){
      numA=100;
   }

   // Declaring class B as a friend
   friend class B;
};
```

```
class B {
public:
   void display(A obj) {
      // Accessing private member of class A
      cout << "Value of numA: " << obj.numA << endl;
   }
};

int main() {
   A objA;
   B objB;
   objB.display(objA); // B can access private data of A
   return 0;
}
```

Output:

```
Value of numA: 100
```

Benefits of Friend Classes

- **Allows direct access to private data –** Useful for tightly coupled classes.
- **Maintains Encapsulation –** Only the specific class gets access, rather than making the data public.
- **Better Performance –** Avoids unnecessary getter/setter functions.
- **Useful for Operator Overloading –** Helps in overloading operators that need access to private data.

Aggregation

- Aggregation is a type of association that represents a "has-a" relationship between two classes. It’s a way to create complex classes by combining simpler ones.
  - One class (the container) contains objects of another class (the contained).
  - The lifetime of the contained objects is independent of the container. (If the container is destroyed, the contained objects can still exist.)

  Example:
  - Let’s say we have a Department and an Employee class. A department has employees, but employees can exist even if the department is dissolved.

Example

```
#include <iostream>
using namespace std;

class Engine {
public:
    void start() {
        cout << "Engine started!" << endl;
    }
};

class Car {
private:
    Engine* engine;  // Aggregation: Car has an Engine
public:
    Car(Engine* eng){
        engine=eng;
    }
    void startCar() {
        engine->start();  // Using Engine's functionality
        cout << "Car is running!" << endl;
    }
};

int main() {
    Engine eng;  // Engine object created independently
    Car car(&eng);  // Car has an Engine

    car.startCar(); // Starting the car

    return 0;
}
```

Output:

```
Engine started!
Car is running!
```

Base Classes and Derived Classes

- **Base Class**
  - A Base Class is a class that provides attributes and behaviors (methods) that can be inherited by other classes (Derived Classes). It serves as a parent class in inheritance.

- **Derived Class**
  - A Derived Class is a class that inherits properties and methods from a Base Class. It allows code reuse and follows an "is-a" relationship.

Example

```
#include <iostream>
using namespace std;

// Base class
class Animal {
public:
    void eat() {
        cout << "This animal eats food." << endl;
    }
};

// Derived class
class Dog : public Animal {
public:
    void bark() {
        cout << "The dog barks!" << endl;
    }
};

int main() {
    Dog myDog;
    myDog.eat();  // Inherited from Animal (Base Class)
    myDog.bark(); // Own function

    return 0;
}
```

Output:

```
This animal eats food.
The dog barks!
```

Inheritance

- Inheritance is a fundamental concept in C++ that allows a Derived Class to inherit properties and methods from a Base Class. It enables code reusability and establishes a relationship between classes.

- **Types of Inheritance in C++**
  - **Single Inheritance** – One class inherits from another.
  - **Multiple Inheritance** – A class inherits from more than one base class.
  - **Multilevel Inheritance** – A derived class inherits from another derived class.
  - **Hierarchical Inheritance** – Multiple derived classes inherit from the same base class.
  - **Hybrid Inheritance** – A combination of multiple inheritance types.

Syntax of Inheritance

Syntax:

```
class BaseClass {
   // Base class members
};

class DerivedClass : public BaseClass {
   // Derived class members
};
```

Single Inheritance

- Single inheritance is a type of inheritance in which a derived class inherits from only one base class. This is the simplest form of inheritance and is widely used to promote code reuse.

- Key Points:
  - **Simple Structure** – The derived class inherits only from one base class.
  - **Code Reusability** – The derived class can reuse the functionality of the base class.
  - **No Ambiguity** – Since there’s only one base class, there's no ambiguity in method resolution.

Example

```
#include <iostream>
using namespace std;

// Base class (Parent)
class Animal {
public:
    void eat() {
        cout << "This animal eats food." << endl;
    }
};

// Derived class (Child)
class Dog : public Animal {
public:
    void bark() {
        cout << "The dog barks!" << endl;
    }
};

int main() {
    Dog myDog;
    myDog.eat();  // Inherited from Animal
    myDog.bark(); // Defined in Dog class

    return 0;
}
```

Output:

```
This animal eats food.
The dog barks!
```

Multiple Inheritance

- Multiple inheritance is a type of inheritance where a derived class inherits from more than one base class. This allows the derived class to access members from multiple parent classes.

- Key Points of Multiple Inheritance:
  - **Code Reusability** – A derived class can reuse functionalities from multiple base classes.
  - **Combining Features** – It allows a class to combine features from different sources.
  - **Ambiguity Issue** – If both base classes have a function with the same name, the derived class must specify which one to use using the scope resolution operator (::).

Example

```
#include <iostream>
using namespace std;

// Base class 1
class Animal {
public:
   void eat() {
      cout << "This animal eats food." << endl;
   }
};

// Base class 2
class Bird {
public:
   void fly() {
      cout << "This bird can fly." << endl;
   }
};

// Derived class inheriting from both Animal and Bird
class Bat : public Animal, public Bird {
public:
   void sleep() {
      cout << "This bat sleeps in the day." << endl;
   }
};

int main() {
   Bat myBat;
   myBat.eat();  // Inherited from Animal
   myBat.fly();  // Inherited from Bird
   myBat.sleep(); // Defined in Bat

   return 0;
}
```

Output:

```
This animal eats food.
This bird can fly.
This bat sleeps in the day.
```

Multilevel Inheritance

- Multilevel inheritance is a type of inheritance where a class is derived from another derived class. This forms a chain of inheritance, where each level passes its properties to the next.

- **Key Points of Multilevel Inheritance:**
  - **Code Reusability** – Each level reuses features of its parent.
  - **Hierarchy Representation** – It models real-world hierarchies (e.g., Animal → Mammal → Dog).
  - **Can Lead to Complexity** – If inheritance is too deep, it can make the code hard to manage.

Example

```
#include <iostream>
using namespace std;

// Base class (Grandparent)
class Animal {
public:
    void eat() {
        cout << "This animal eats food." << endl;
    }
};

// Derived class (Parent)
class Mammal : public Animal {
public:
    void breathe() {
        cout << "This mammal breathes air." << endl;
    }
};

// Derived class (Child)
class Dog : public Mammal {
public:
    void bark() {
        cout << "The dog barks!" << endl;
    }
};

int main() {
    Dog myDog;
    myDog.eat();    // Inherited from Animal
    myDog.breathe(); // Inherited from Mammal
    myDog.bark();    // Defined in Dog

    return 0;
}
```

Output:

```
This animal eats food.
This mammal breathes air.
The dog barks!
```

Hierarchical Inheritance

- Hierarchical inheritance is a type of inheritance where multiple derived classes inherit from a single base class. This allows multiple classes to share the properties and behavior of a common base class.

- **Key Points of Hierarchical Inheritance:**
  - **Code Reusability** – The base class (Animal) provides common functionality that all derived classes (Dog, Cat) can use.
  - **Independent Derived Classes** – Each derived class has its own specialized behavior.
  - **Avoids Code Duplication** – Instead of writing eat() in Dog and Cat, both inherit it from Animal.

Example

```
#include <iostream>
using namespace std;

// Base class (Parent)
class Animal {
public:
    void eat() {
        cout << "This animal eats food." << endl;
    }
};

// Derived class 1 (Child)
class Dog : public Animal {
public:
    void bark() {
        cout << "The dog barks!" << endl;
    }
};

// Derived class 2 (Child)
class Cat : public Animal {
public:
    void meow() {
        cout << "The cat meows!" << endl;
    }
};

int main() {
    Dog myDog;
    myDog.eat();  // Inherited from Animal
    myDog.bark(); // Defined in Dog

    Cat myCat;
    myCat.eat();  // Inherited from Animal
    myCat.meow(); // Defined in Cat

    return 0;
}
```

Output:

```
This animal eats food.
The dog barks!
This animal eats food.
The cat meows!
```

Hybrid Inheritance

- Hybrid inheritance is a combination of two or more types of inheritance in a single program. It can mix single, multiple, multilevel, or hierarchical inheritance, depending on the scenario.

Example

```
#include <iostream>
using namespace std;

// Base class
class Person {
public:
   void showPerson() {
      cout << "This is a person." << endl;
   }
};

// First derived class (Single Inheritance)
class Student : public Person {
public:
   void showStudent() {
      cout << "This person is a student." << endl;
   }
};

// Second base class for Multiple Inheritance
class Teacher {
public:
   void showTeacher() {
      cout << "This person is a teacher." << endl;
   }
};

// Derived class inheriting from both Student and Teacher (Multiple Inheritance)
class TeachingAssistant : public Student, public Teacher {
public:
   void showTA() {
      cout << "This person is a Teaching Assistant." << endl;
   }
};

int main() {
   TeachingAssistant ta;

   ta.showPerson();  // Inherited from Person (Hierarchical)
   ta.showStudent(); // Inherited from Student (Multilevel)
   ta.showTeacher(); // Inherited from Teacher (Multiple)
   ta.showTA();      // Own function

   return 0;
}
```

Output:

```
This is a person.
This person is a student.
This person is a teacher.
This person is a Teaching Assistant.
```

Public, private and protected in Inheritance

- In C++, the access specifiers (public, private, protected) determine how members of the base class are inherited in the derived class.

Public Inheritance (public)

- public members remain public in the derived class.
- protected members remain protected in the derived class.
- private members are not directly accessible in the derived class.

Example

```
#include <iostream>
using namespace std;

class Base {
public:
   int a = 10;
protected:
   int b = 20;
private:
   int c = 30;  // Not inherited
};

class Derived : public Base {
public:
   void show() {
      cout << "Public a: " << a << endl;   // ✅ Accessible
      cout << "Protected b: " << b << endl; // ✅ Accessible
      // cout << c;  // ❌ Error: Private members are not inherited
   }
};

int main() {
   Derived obj;
   obj.show();
   cout << "Accessing a: " << obj.a << endl;  // ✅ Allowed (public)
   // cout << obj.b;  // ❌ Error: b is protected
   return 0;
}
```

Output:

```
Public a: 10
Protected b: 20
Accessing a: 10
```

Private Inheritance (private)

- public and protected members of the base class become private in the derived class.
- private members are not inherited.

Example

```
#include <iostream>
using namespace std;

class Base {
public:
   int a = 10;
protected:
   int b = 20;
private:
   int c = 30;  // Not inherited
};

class Derived : private Base {
public:
   void show() {
      cout << "Accessing a: " << a << endl;  // ✅ Becomes private in
Derived
      cout << "Accessing b: " << b << endl;  // ✅ Becomes private in
Derived
   }
};

int main() {
   Derived obj;
   obj.show();
   // obj.a; // ❌ Error: a is now private
   return 0;
}
```

Output:

```
Accessing a: 10
Accessing b: 20
```

Private Inheritance (private)

- public and protected members of the base class become private in the derived class.
- private members are not inherited.

Example

```
#include <iostream>
using namespace std;

class Base {
public:
   int a = 10;
protected:
   int b = 20;
private:
   int c = 30;  // Not inherited
};

class Derived : private Base {
public:
   void show() {
      cout << "Accessing a: " << a << endl;  // ✓ Becomes private in Derived
      cout << "Accessing b: " << b << endl;  // ✓ Becomes private in Derived
   }
};

int main() {
   Derived obj;
   obj.show();
   // obj.a; // ✗ Error: a is now private
   return 0;
}
```

Output:

```
Accessing a: 10
Accessing b: 20
```

Constructors and Destructors in Derived Classes

- When a derived class is created, its constructor is responsible for initializing both its own members and the base class members. Similarly, when an object is destroyed, the destructor executes in the reverse order.

- **Order of Execution in Inheritance**
  - Base class constructor runs first.
  - Derived class constructor runs next.
  - Derived class destructor runs first when deleting an object.
  - Base class destructor runs last.

Example

```
#include <iostream>
using namespace std;

class Base {
public:
    Base() {
        cout << "Base class constructor called" << endl;
    }
    ~Base() {
        cout << "Base class destructor called" << endl;
    }
};

class Derived : public Base {
public:
    Derived() {
        cout << "Derived class constructor called" << endl;
    }
    ~Derived() {
        cout << "Derived class destructor called" << endl;
    }
};

int main() {
    Derived obj;  // Object creation triggers constructor chain
    return 0;     // Object destruction triggers destructor chain
}
```

Output:

```
Base class constructor called
Derived class constructor called
Derived class destructor called
Base class destructor called
```

C++ Virtual Destructor

- A **virtual destructor** in C++ ensures that the destructor of the derived class is called when an object is deleted through a base class pointer. This is crucial for proper resource cleanup when dealing with **polymorphism**.

- **Why Use a Virtual Destructor?**
  - When a class has virtual functions, it is generally a good practice to define a **virtual destructor** to ensure correct **destruction of derived class objects** when deleting through a base class pointer.

Example Without a Virtual Destructor (Memory Leak)

```
#include <iostream>

class Base {
public:
    Base() { std::cout << "Base Constructor\n"; }
    ~Base() { std::cout << "Base Destructor\n"; }  // Not virtual
};

class Derived : public Base {
public:
    Derived() { std::cout << "Derived Constructor\n"; }
    ~Derived() { std::cout << "Derived Destructor\n"; } // Not called if deleted
via Base*
};

int main() {
    Base* ptr = new Derived(); // Base pointer pointing to Derived object
    delete ptr; // Only Base's destructor is called!
    return 0;
}
```

Output:

```
Base Constructor
Derived Constructor
Base Destructor
```

Using a Virtual Destructor

```
#include <iostream>

class Base {
public:
    Base() { std::cout << "Base Constructor\n"; }
    virtual ~Base() { std::cout << "Base Destructor\n"; }  // Virtual
destructor
};

class Derived : public Base {
public:
    Derived() { std::cout << "Derived Constructor\n"; }
    ~Derived() { std::cout << "Derived Destructor\n"; }
};

int main() {
    Base* ptr = new Derived();
    delete ptr; // Both destructors are now called!
    return 0;
}
```

Output:

```
Base Constructor
Derived Constructor
Derived Destructor
Base Destructor
```